

Framework comparison — LangGraph vs LangChain vs Google ADK

Three frameworks were evaluated for fitness against the specific shape of this agent: a five-stage sequential pipeline with a retry-on-error conditional edge, requiring per-stage introspection for the UI.

1. Qualitative comparison

DIMENSION	LANGGRAPH	LANGCHAIN (AGENT)	GOOGLE ADK
Fit for staged pipeline	Excellent — explicit graph, first-class nodes/edges	Moderate — implicit control flow via chains	Good — <code>SequentialAgent</code> / custom <code>Agent</code> primitives
Validation-retry loop	First-class — conditional edges native	Manual wiring required	Workable via <code>LoopAgent</code> + custom logic
Observability hooks	Strong — <code>TypedDict</code> state visible; <code>astream_events</code> for per-node lifecycle	Good, via LangSmith	Built-in tracing, but agent-scoped not node-scoped
Per-node event stream for UI live progress	First-class — <code>astream_events(version="v1")</code>	Fine	Streaming exists but not node-granular
Explicit state debugging	Wins — <code>TypedDict</code> state, print at any node	State is embedded in chain internals	State inside <code>Session</code> object; less transparent
Multi-agent hierarchies	Fine — decomposition done in orchestrator layer	Fine via chains	Wins — first-class <code>ParallelAgent</code> , subagent primitives
Voice / multimodal	Not supported	Not supported	Wins — native bidirectional audio, live sessions
Ecosystem maturity	Wins — 2+ years of examples, integrations	Largest — LangChain family is the biggest ecosystem	Younger, growing fast, Google-specific
Managed deployment	Self-host	Self-host	Wins — Vertex AI Agent Engine
Model coupling	Wins — model-agnostic	Wins — model-agnostic	Gemini-first, others via LiteLLM

Best when	Controlled multi-step agents with explicit state + retry	Fast prototyping with chain compositions	Voice-first, multi-agent, or Google-ecosystem shops
-----------	--	--	---

2. Why LangGraph was chosen for this build

Three properties of this specific agent matched LangGraph's strengths and would have required workarounds in the alternatives.

2.1 The retry conditional edge

The pipeline has one non-trivial control-flow requirement: on Validate or Execute failure, route back to Generate with the error string injected as a correction hint, capped at 3 attempts, then graceful exit.

- **In LangGraph:** $\approx$ 15 lines of `add_conditional_edges` + a `_maybe_retry` helper. Fits the primitive.
- **In LangChain:** would need to compose chains with manual retry decorators; the error-injection-as-prompt-hint pattern requires custom chain wiring.
- **In ADK:** doable via `LoopAgent` with a custom termination condition, but the loop-control semantics are less native to a "retry on specific error class with error injected into next prompt" pattern.

2.2 Per-stage live progress via SSE

The frontend shows a 5-stage progress pill that ticks in real time. This requires the framework to emit lifecycle events at each pipeline stage boundary.

- **LangGraph's** `astream_events(version="v1")` emits `on_chain_start` and `on_chain_end` for every node, which map 1-to-1 to the frontend's `stage_start` / `stage_end` events. Zero translation layer.
- **ADK's event stream** is agent-scoped rather than node-scoped; wrapping it to match this granularity would have added a translation layer.
- **LangChain** streaming is chain-scoped; per-step lifecycle events would need custom callback handlers.

2.3 Provider portability

The system runs on OpenAI GPT-4o. LangGraph is model-agnostic; ADK is Gemini-first (others via LiteLLM adapter). No wrapper needed with LangGraph; ADK would have required an extra abstraction layer to keep the code portable.

3. Why the comparison is qualitative, not measured

The client's brief mentioned framework comparison but did not require a measured head-to-head port.

Building a full ADK port of the same 5-node pipeline (to produce measured latency / token / accuracy numbers) was **scoped and deferred** as an above-baseline flex. The timebox was better spent on the interactive dashboard (see 06_EXTENSION_ROADMAP). The

`FRAMEWORK=langgraph|adk` env var scaffold is in `backend/app/config.py` if the port is picked up later.

What we do have: the LangGraph implementation itself, running against the eval harness — see 04_EVALUATION . Measured latency, accuracy, cost, and safety-catch rates for the framework the client will actually receive.

What we don't have: measured numbers for ADK or LangChain implementations of the same pipeline.

An honest reader can verify LangGraph's fitness for this workload from the measured numbers. Whether ADK or LangChain would beat it on the same workload requires porting; that's future work.

4. Recommendation

For this workload class (NL→SQL business analytics with a controlled multi-step pipeline):
LangGraph.

The three properties in §2 make it the correct choice today, and the numbers in 04_EVALUATION show it meets or exceeds every target.

Reconsider if:

- The product pivots to voice-first or multimodal → **Google ADK** (native audio streaming).
- The team adopts Vertex AI Agent Engine for managed deployment → **Google ADK** (managed runtime with sessions, memory, tracing).
- Multi-agent orchestration becomes a core feature — e.g. dedicated Analyst / Charter / Reporter subagents delegating to each other → **Google ADK** (first-class subagent primitives).
- The application drops the graph model and becomes a series of loosely-connected chains → **LangChain** (chain composition is its native surface).

5. What each framework is genuinely best at

Independent of this specific build, here's the honest sweet spot of each:

LangGraph

Best for: state machines that happen to run LLMs. When you want fine control over an orchestration graph with explicit state and conditional flow. When per-node introspection is a first-class concern (debuggers, live UI progress, replay).

Weakest at: multi-agent hierarchies (subagents calling subagents — doable but manual), voice/multimodal, managed deployment.

LangChain

Best for: rapid prototyping with many chain compositions. When the application is a series of "take X, transform to Y, hand to Z" steps rather than a state machine with feedback loops.

Weakest at: structured retry loops with error injection, per-step introspection at the granularity a UI needs.

Google ADK

Best for: hierarchies of specialised agents (planner + worker + reviewer patterns). Voice-first agents (native bidirectional audio). Google-ecosystem-heavy stacks (Gemini + Vertex).

Weakest at: state transparency (state lives inside Session objects, less easy to poke than a plain dict), per-node event stream at UI granularity, model portability (Gemini-first bias).

6. Design decision reference

The decision to use LangGraph is captured in `architecture/DECISIONS` as D-005 (`TypedDict` state), D-006 (thin `llm.py` wrapper avoiding LangChain surface), and adjacent entries. The retry loop's use of LangGraph's conditional edges is D-022. Streaming architecture built on `astream_events` is D-029.